

Integration Guide

A CEREBRUM Research Publication (v1.2.4)

Bryan Alexander Buchorn
Independent Researcher

Abstract

Overview: This guide provides a conceptual entry point into the CEREBRUM framework. It is designed for practitioners and researchers seeking to understand the core reasoning signals of Framework Implementation Guide.

0.1 CEREBRUM Integration Guide

Version: v1.1.0 **Audience:** Python developers integrating CEREBRUM into an application

This guide covers the Python API directly. For the REST API, see `docs/API_REFERENCE.md`. For deployment, see `docs/DEPLOYMENT.md`.

—

Installation

```
# Core only (no sentence-transformers, no API server)
pip install -e "."

# With semantic embeddings
pip install -e "[embeddings]"

# With REST API server
pip install -e "[api]"

# Everything
pip install -e "[all]"
```

—

1. Loading a Graph

From CSV

```
from adapters.csv_adapter import load_csv_adapter

adapter = load_csv_adapter("path/to/graph.csv")
# CSV format: subject,predicate,object[,weight]
# Header row optional; auto-detected
```

From NetworkX

```
import networkx as nx
from adapters.networkx_adapter import NetworkXAdapter

G = nx.DiGraph()
G.add_edge("Marie Curie", "Radium", relation="discovered", weight=0.95)
G.add_edge("Radium", "Radioactivity", relation="property_of", weight=0.90)

adapter = NetworkXAdapter(G)
```

From Neo4j

```
from adapters.neo4j_adapter import Neo4jAdapter

adapter = Neo4jAdapter(
    uri="bolt://localhost:7687",
    user="neo4j",
    password=os.environ["NEO4J_PASSWORD"],
)
```

From RDF / SPARQL

```
from adapters.rdf_adapter import RDFAdapter

adapter = RDFAdapter(sparql_endpoint="https://dbpedia.org/sparql")
```

Programmatic construction with IngestionPipeline

```
from core.thalamus import IngestionPipeline

pipeline = IngestionPipeline(
    adapter,
    namespace="text",          # Phase 19: isolate from signal entities
    entity_dedup_map={         # alias normalization
        "M. Curie": "Marie Curie",
        "Madame Curie": "Marie Curie",
    }
)

pipeline.process("Marie Curie", "discovered", "Radium", confidence=0.99,
    source="wikipedia")
pipeline.process("Radium", "causes", "Radiation_Sickness", confidence=0.85,
    source="pubmed")
```

—

2. Community Detection (DSCF)

```
from core.community_engine import best_of_n_dscf, modularity_score

G = adapter.to_networkx()

# Best of N trials (recommended: n_trials=5 for reproducibility)
partitions = best_of_n_dscf(G, n_trials=5, seed=42)
q = modularity_score(G, partitions)
print(f"{len(partitions)} communities, Q={q:.4f}")

# Build community_map: {node_id: community_id}
community_map = {}
for cid, members in enumerate(partitions):
    for node in members:
        community_map[node] = cid
```

Algorithm selection

```
from core.community_engine import CommunityEngine

# DSCF (default, recommended)
engine = CommunityEngine(adapter, algorithm="dscf")

# Leiden (native, GPL-free)
engine = CommunityEngine(adapter, algorithm="leiden")

# LPA only (fast, lower quality)
engine = CommunityEngine(adapter, algorithm="lpa")

# With soft membership (Phase 17)
engine = CommunityEngine(adapter, algorithm="dscf", soft_membership=True, tau
    =2.0)
```

—

3. Embeddings

```
from core.embedding_engine import RandomEngine, SentenceEngine

# Random (fast, no external deps, good for structural reasoning)
engine = RandomEngine(dim=64, seed=42)
embeddings = engine.encode_entities({node: node for node in G.nodes()})

# Semantic (requires: pip install -e "[embeddings]")
engine = SentenceEngine(model="all-MiniLM-L6-v2")
embeddings = engine.encode_entities({node: node for node in G.nodes()})

# KGE embeddings (Phase 17, requires training)
from core.kge_engine import RotateEngine, KGEEmbeddingAdapter
kge = RotateEngine(adapter, dim=128, epochs=200)
kge.train()
```

```
embeddings = KGEmbeddingAdapter(kge)
```

4. Building the CSA Engine

```
from core.attention_engine import CSAEngine
from core.structural_encoder import build_community_distance_matrix,
    adjacent_community_pairs

dist = build_community_distance_matrix(G, community_map)
adj = adjacent_community_pairs(G, community_map)

csa = CSAEngine(
    communities=community_map,
    embeddings=embeddings,
    # Optional: per-community parameter overrides (Phase 20)
    community_params={
        3: (0.5, 0.15, 0.25, 0.05, 0.05, 0.0), # protein community    reduce
            , increase
    },
)
csa.set_community_graph(dist, adj)

# Phase 20: snapshot isolation for concurrent use
csa.set_query_snapshot(community_map)
```

5. Running a Query

```
from reasoning.traversal import BeamTraversal
from reasoning.answer_extractor import extract

traversal = BeamTraversal(
    adapter=adapter,
    csa_engine=csa,
    embeddings=embeddings,
    communities=community_map,
    beam_width=10,
    max_hop=3,
    # Optional: Bayesian mode (Phase 18)
    probabilistic=True,
    warm_start_strength=5.0, # Phase 19: reduce cold-start variance
)

paths = traversal.traverse(["Marie Curie"])
answers = extract(paths, top_k=5)

for ans in answers:
    print(f"{ans.entity_id:30s} score={ans.score:.4f}")
    print(f" path: {' -> '.join(ans.best_path.nodes)}")
```

6. LLM Bridge (Optional)

Convert CEREBRUM reasoning results into prompts or structured context for any LLM:

```
from llm_bridge import generate, to_prompt
from llm_bridge import AnthropicAdapter # or OpenAIAdapter, OllamaAdapter,
    HuggingFaceAdapter

# Format as a prompt string
prompt = to_prompt(answers, query="What did Marie Curie discover?")

# Generate an LLM response grounded in the graph paths
result = generate(
    answers=answers,
    query="What did Marie Curie discover?",
    adapter=AnthropicAdapter(api_key=os.environ["ANTHROPIC_API_KEY"], model="
        claude-sonnet-4-6"),
)

print(result.text)
print(result.grounded_paths) # The CEREBRUM paths the LLM was given as
    context
```

See `llm_bridge/README.md` for adapter configuration details.

—

7. Streaming Integration

```
from adapters.stream_adapter import StreamAdapter
from core.discretizer import STDPDiscretizer, ThresholdDiscretizer
from core.rebalancer import GlobalRebalancer

# Wrap existing adapter with streaming
stream = StreamAdapter(
    base_adapter=adapter,
    window_size=10_000,
    batch_size=50,
)

# Add discretizers
stream.add_discretizer(STDPDiscretizer(
    w_threshold=0.5, n_min=5,
    min_causal_span=1.0, # adversarial hardening
))

# Background rebalancer
rebalancer = GlobalRebalancer(
    adapter=stream,
    q_drift_threshold=0.05,
    bridge_engine=bridge_engine, # optional
)
rebalancer.start()

# Push events
```

```
stream.push_event("sensor_a", "READS", "temp_42", timestamp=time.time(),
weight=0.8)
```

—

8. Federated Deployment

```
from adapters.remote_adapter import RemoteAdapter
from adapters.federated_adapter import FederatedAdapter

# Connect to remote CEREBRUM nodes
node_a = RemoteAdapter("https://node-a.example.com", token=os.environ["
    NODE_A_TOKEN"])
node_b = RemoteAdapter("https://node-b.example.com", token=os.environ["
    NODE_B_TOKEN"])

# Create federated view
federated = FederatedAdapter([local_adapter, node_a, node_b])

# Query traverses all nodes transparently
traversal = BeamTraversal(adapter=federated, csa_engine=csa, ...)
```

—

9. Complete Minimal Example

```
from adapters.csv_adapter import load_csv_adapter
from core.community_engine import best_of_n_dscf
from core.embedding_engine import RandomEngine
from core.attention_engine import CSAEngine
from core.structural_encoder import build_community_distance_matrix,
    adjacent_community_pairs
from reasoning.traversal import BeamTraversal
from reasoning.answer_extractor import extract

adapter = load_csv_adapter("tests/fixtures/toy_graph.csv")
G        = adapter.to_networkx()

parts    = best_of_n_dscf(G, n_trials=5, seed=42)
cmap     = {node: cid for cid, members in enumerate(parts) for node in members}

emb      = RandomEngine(dim=64).encode_entities({n: n for n in G.nodes()})
csa      = CSAEngine(communities=cmap, embeddings=emb)
csa.set_community_graph(
    build_community_distance_matrix(G, cmap),
    adjacent_community_pairs(G, cmap),
)
csa.set_query_snapshot(cmap)

traversal = BeamTraversal(adapter=adapter, csa_engine=csa, embeddings=emb,
    communities=cmap, beam_width=10, max_hop=3)
answers   = extract(traversal.traverse(["newton"]), top_k=5)
```

```
for a in answers:  
    print(f"{a.entity_id:25s}  {a.score:.4f}  {' -> '.join(a.best_path.nodes)}")
```

– Copyright © 2026 Bryan Alexander Buchorn. All Rights Reserved.

Project CEREBRUM — March 2026 — Hardened Enterprise Security (v1.2.4)